

Лекция 2. Хранение данных. Модель SQL, NoSQL и анализ запросов

Технологии и инструментарий анализа больших данных

Курс ТИАБД

Раздел

Введение

От реляционного хранения к многоформатной
архитектуре данных

Учебная цель лекции

После лекции студент должен уметь объяснить:

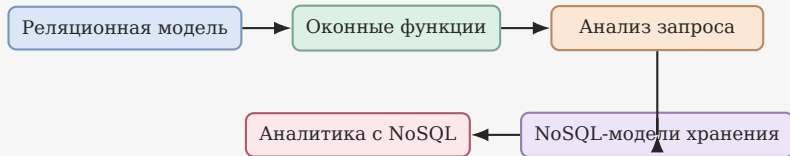
- чем реляционная модель удобна для структурированных данных;
- какую роль играют **DDL**, **DML**, **TCL** в жизненном цикле базы;
- как строятся оконные функции и чем они отличаются от GROUP BY;
- как читать план SQL-запроса через EXPLAIN;
- зачем возникли NoSQL-системы и какие модели хранения они используют;
- как выбирать систему хранения под прикладной сценарий;
- как NoSQL-данные включаются в аналитический pipeline.

Главный тезис

Система хранения выбирается не по названию технологии, а по структуре данных, запросам, требованиям к задержке, согласованности и масштабированию.

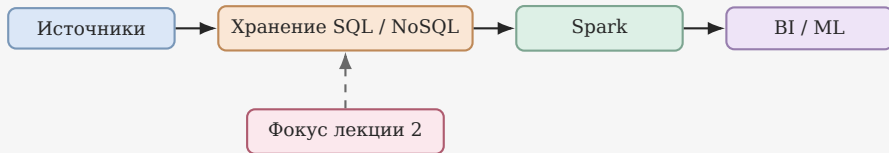


Логика лекции



SQL-блок будет компактным напоминанием. Больше времени уделяется двум темам: **оконной аналитике** и **NoSQL как семейству моделей хранения**.

Место лекции в курсе



Связь с последующими темами

В третьей лекции Spark будет читать данные из файлов, таблиц и внешних систем хранения. Поэтому уже сейчас нужно понимать, **какая модель данных находится под аналитической задачей.**

Раздел

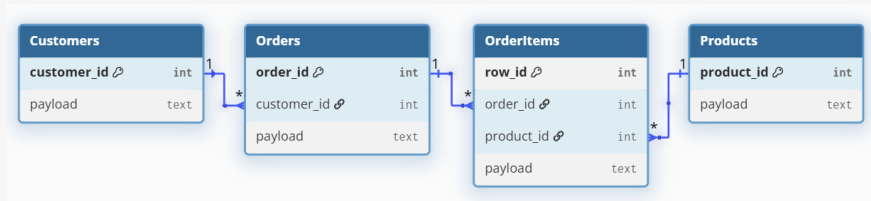
Реляционная модель и SQL

Краткое напоминание реляционной модели

Реляционная модель данных

Определение

Реляционная модель представляет данные в виде отношений, то есть таблиц с фиксированным набором атрибутов, типами данных и ограничениями целостности.



- строка — кортеж предметной области;
- столбец — атрибут с типом;
- первичный ключ идентифицирует строку;
- внешний ключ фиксирует связь между таблицами.

Почему реляционная модель остаётся важной

Сильные стороны

- строгая схема;
- ограничения целостности;
- транзакции;
- декларативный SQL;
- развитая оптимизация запросов;
- зрелые инструменты BI.

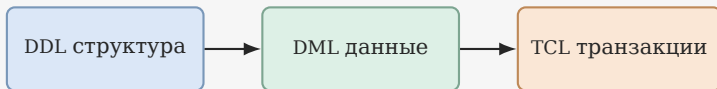
Ограничения

- фиксированная структура;
- дорогие JOIN на больших объёмах;
- сложность горизонтального масштабирования;
- не всегда удобна для вложенных JSON, графов, потоков событий;
- схема меняется медленнее продукта.

Вывод

Реляционные СУБД не исчезают из Big Data-архитектур. Они становятся одним из слоёв полиглотного хранения.

DDL, DML, TCL



Класс	Команды	Назначение
DDL	CREATE, ALTER, DROP	создать и изменить схему
DML	SELECT, INSERT, UPDATE, DELETE	прочитать и изменить данные
TCL	BEGIN, COMMIT, ROLLBACK	управлять границами транзакций

DDL структура данных

```
CREATE TABLE orders (  
  order_id    text PRIMARY KEY,  
  customer_id text NOT NULL,  
  order_date  date NOT NULL,  
  status      text NOT NULL  
);  
  
CREATE TABLE order_items (  
  order_id    text NOT NULL,  
  product_id  text NOT NULL,  
  price       numeric(12, 2) NOT NULL,  
  quantity    integer NOT NULL,  
  PRIMARY KEY (order_id, product_id)  
);
```

Зачем нужен DDL

DDL фиксирует типы, ключи и ограничения, то есть **контракт** (схема на запись) между источниками данных, приложением и аналитикой.



DML чтение и изменение данных

```
INSERT INTO orders(order_id, customer_id, order_date, status)
VALUES ('o_001', 'c_001', DATE '2026-09-01', 'delivered');
```

```
SELECT customer_id, COUNT(*) AS orders_cnt
FROM orders
WHERE status = 'delivered'
GROUP BY customer_id
ORDER BY orders_cnt DESC;
```

- INSERT, UPDATE, DELETE обслуживают операционную часть;
- SELECT становится основой аналитики;
- аналитический SQL быстро выходит за пределы простых выборок и требует оконных функций, CTE и анализа планов.



TCL транзакционная граница

```
BEGIN;
```

```
UPDATE inventory  
SET stock = stock - 1  
WHERE product_id = 'p_001';
```

```
INSERT INTO orders(order_id, customer_id, order_date, status)  
VALUES ('o_002', 'c_010', CURRENT_DATE, 'created');
```

```
COMMIT;
```

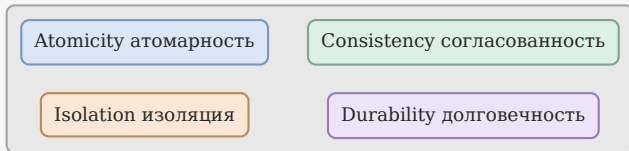
```
-- или ROLLBACK при ошибке
```

Зачем нужен TCL

Транзакция связывает несколько изменений в одну логическую операцию. Для OLTP это критично к использованию, поскольку заказ не должен появиться без корректного изменения остатков, платежа или статуса.



ACID как свойство транзакционной СУБД



транзакционная модель

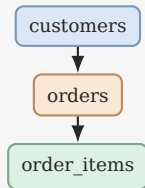
- атомарность: операция либо выполнена целиком, либо не выполнена;
- согласованность: ограничения схемы не нарушаются;
- изоляция: параллельные транзакции не должны разрушать смысл данных;
- долговечность: зафиксированные изменения переживают сбой.



Нормализация и аналитическая денормализация

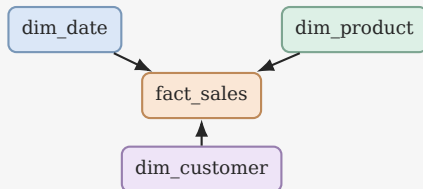
OLTP-логика

Нормализация уменьшает избыточность и защищает целостность.



OLAP-логика

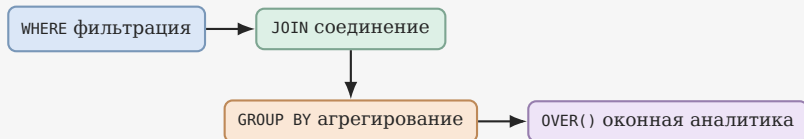
Денормализация ускоряет повторяемую аналитику и BI.



То, что хорошо для записи и целостности, не всегда оптимально для тяжёлой аналитики.

Минимум SQL перед аналитикой

Для дальнейших разделов достаточно удерживать четыре базовые идеи.



Переход

GROUP BY отвечает на вопрос «какова агрегированная характеристика группы?». Оконная функция отвечает на вопрос «каково положение или агрегат строки внутри её аналитического контекста?».

Раздел

Оконные функции

Аналитика строки в контексте группы

Проблема обычного агрегирования

Наблюдение

GROUP BY уменьшает детализацию и несколько строк превращаются в одну строку результата.



Но в аналитике часто нужно сохранить каждую строку и добавить к ней контекст:

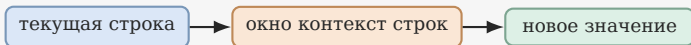
- номер заказа клиента;
- долю товара в продажах категории;
- накопительную сумму продаж;
- сравнение текущего значения с предыдущим;
- ранжирование внутри сегмента.



Что такое оконная функция

Определение

Оконная функция вычисляет значение для текущей строки на основе множества строк, связанных с ней аналитическим окном, при этом не схлопывает строки результата.



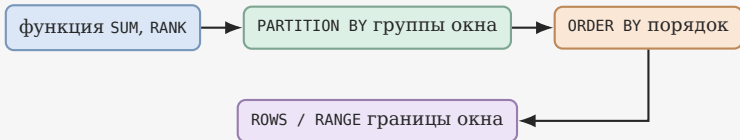
$$\text{result row} = \text{source row} + f(\text{window})$$

Строка остаётся в результате, но получает дополнительный аналитический признак.



Синтаксис оконной функции

```
<function>() OVER (  
  PARTITION BY <columns>  
  ORDER BY <columns>  
  ROWS BETWEEN <start> AND <end>  
)
```



Правило чтения

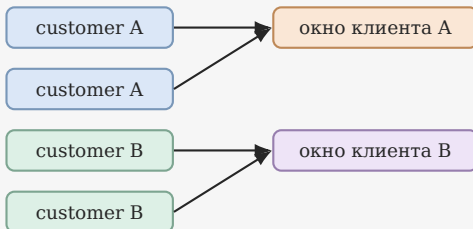
Сначала определяем, какие строки входят в аналитический контекст, затем задаём порядок, затем вычисляем функцию для каждой строки.



PARTITION BY логические группы окна

Принцип

PARTITION BY разбивает результат запроса на независимые аналитические группы. Окно текущей строки не выходит за пределы её partition.

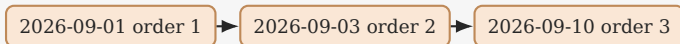


Примеры partition: клиент, товарная категория, город, продавец, дата, пользовательская сессия.

ORDER BY порядок внутри окна

Принцип

ORDER BY внутри OVER() задаёт аналитический порядок строк, устанавливая хронологию, убывание выручки, последовательность событий.



ORDER BY order_date

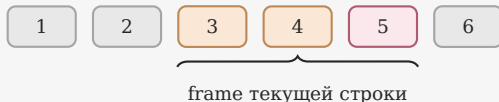
- для рангов порядок определяет победителя;
- для накопительных сумм порядок определяет накопление;
- для LAG/LEAD порядок определяет предыдущую и следующую строку.



Frame границы вычисления

Определение

Window frame — подмножество строк внутри partition, реально используемое функцией для текущей строки.



Типичные границы:

- UNBOUNDED PRECEDING — от начала partition;
- CURRENT ROW — до текущей строки;
- 1 PRECEDING — одна предыдущая строка;
- 1 FOLLOWING — одна следующая строка.

ROW_NUMBER порядковый номер

```
SELECT
    customer_id,
    order_id,
    order_date,
    ROW_NUMBER() OVER (
        PARTITION BY customer_id
        ORDER BY order_date
    ) AS customer_order_num
FROM orders;
```

Интерпретация

Для каждого клиента заказы сортируются по дате. Каждому заказу присваивается номер внутри истории этого клиента.

$\text{order}_{i,c} \rightarrow \text{position in sequence of customer } c$



RANK и DENSE_RANK

```
SELECT
    category,
    product_id,
    revenue,
    RANK() OVER (
        PARTITION BY category
        ORDER BY revenue DESC
    ) AS revenue_rank
FROM product_revenue;
```

RANK()

- одинаковым значениям даёт одинаковый ранг;
- после равенства оставляет пропуск.

DENSE_RANK()

- одинаковым значениям даёт одинаковый ранг;
- пропусков в последовательности рангов нет.



RANK против DENSE_RANK: пример

Товар	Выручка	RANK	DENSE_RANK
A	1000	1	1
B	900	2	2
C	900	2	2
D	700	4	3

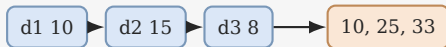
Практический выбор

Если важно место в соревновании — используем RANK. Если нужна компактная нумерация групп равных значений — DENSE_RANK.



Накопительная сумма

```
SELECT
  order_date,
  revenue,
  SUM(revenue) OVER (
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_revenue
FROM daily_sales;
```



Накопительный итог удобен для продаж, числа пользователей, расходов, событий и мониторинговых метрик.



Скользящее среднее

```
SELECT
  order_date,
  revenue,
  AVG(revenue) OVER (
    ORDER BY order_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS moving_avg_7d
FROM daily_sales;
```

Идея

Для каждой даты строится окно из текущего дня и шести предыдущих строк. Среднее сглаживает случайные колебания временного ряда.

$$\bar{x}_t = \frac{1}{7} \sum_{i=t-6}^t x_i$$



order_date	revenue	moving_avg_7d
2016-09-19	194,47	194,47
2016-10-08	908,29	551,38
2016-10-09	1545,57	882,78
2016-10-10	7607,91	2564,06
2016-10-11	7350,06	3521,26
2016-10-12	5160,31	3794,44
2016-10-13	1818,15	3512,11
2016-10-14	5767,93	4308,32
2016-10-15	2400,16	4521,44
2016-10-16	1658,59	4537,59

LAG и LEAD сравнение с соседними строками

```
SELECT
  order_date,
  revenue,
  LAG(revenue) OVER (ORDER BY order_date) AS prev_revenue,
  revenue - LAG(revenue) OVER (ORDER BY order_date) AS delta
FROM daily_sales;
```



Такие функции заменяют громоздкие *self-join* в задачах анализа последовательностей.

order_date	revenue	prev_revenue	delta
2016-09-19	194,47		
2016-10-08	908,29	194,47	713,82
2016-10-09	1545,57	908,29	637,28
2016-10-10	7607,91	1545,57	6062,34
2016-10-11	7350,06	7607,91	-257,85
2016-10-12	5160,31	7350,06	-2189,75
2016-10-13	1818,15	5160,31	-3342,16
2016-10-14	5767,93	1818,15	3949,78
2016-10-15	2400,16	5767,93	-3367,77
2016-10-16	1658,59	2400,16	-741,57

Доля строки в группе

```
SELECT
  category,
  product_id,
  revenue,
  revenue / SUM(revenue) OVER (PARTITION BY category) AS share_in_category
FROM product_revenue;
```

Интерпретация

В каждой строке есть выручка товара и сумма выручки всей категории. Деление даёт долю товара внутри категории без потери детализации.

$$\text{share}_{p,c} = \frac{\text{revenue}_{p,c}}{\sum_{j \in c} \text{revenue}_{j,c}}$$



Оконная функция против GROUP BY

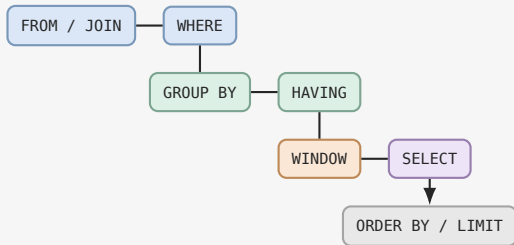
Критерий	GROUP BY	Оконная функция
Число строк	уменьшается	обычно сохраняется
Контекст	группа агрегирования	окно текущей строки
Тип задач	итог по группе	ранги, доли, динамика, сравнение
Детализация	теряется	сохраняется

Ключевая мысль

Оконные функции превращают SQL из языка отчётных агрегатов в язык аналитических признаков, пригодных для BI, ML и временного анализа.



Логический порядок выполнения SQL и окна



- окно вычисляется после фильтрации и группировки;
- оконная функция не может напрямую использоваться в WHERE;
- для фильтрации по рангу обычно используют CTE или подзапрос.



Фильтрация по результату оконной функции

```
WITH ranked_products AS (  
  SELECT  
    category,  
    product_id,  
    revenue,  
    ROW_NUMBER() OVER (  
      PARTITION BY category  
      ORDER BY revenue DESC  
    ) AS rn  
  FROM product_revenue  
)  
SELECT *  
FROM ranked_products  
WHERE rn <= 3;
```

Шаблон

Сначала в СТЕ вычисляется аналитический признак, затем внешний запрос фильтрует строки по этому признаку.



Типовые ошибки в оконных функциях

- забыть `PARTITION BY` и посчитать рейтинг по всей таблице вместо группы;
- забыть `ORDER BY` там, где нужен порядок;
- не указать `frame` для накопительных и скользящих показателей;
- путать `ORDER BY` внутри `OVER()` и финальный `ORDER BY` результата;
- пытаться использовать оконную функцию в `WHERE` без `CTE`;
- считать, что оконная функция всегда быстрее `JOIN` или агрегации.

Инженерный принцип

Оконная функция повышает выразительность SQL, но её стоимость всё равно определяется сортировками, объёмом `partition` и планом выполнения.



Раздел

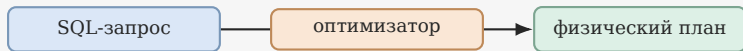
Анализ SQL-запроса

От декларативного текста к физическому плану

SQL декларативен

Тезис

SQL описывает **что** нужно получить, но не описывает в явном виде, **как** именно СУБД должна это вычислить.



Один и тот же результат может быть получен через разные алгоритмы, такие как полное сканирование, индексный доступ, hash join, nested loop, сортировку или предварительную агрегацию.



Зачем анализировать план запроса

- найти полные сканирования больших таблиц;
- понять, используются ли индексы;
- увидеть тип JOIN и его стоимость;
- обнаружить дорогие сортировки и агрегации;
- сравнить ожидаемое и фактическое число строк;
- проверить влияние CTE, подзапроса, оконной функции;
- оценить, почему запрос медленный до переписывания SQL.

Главный навык

Аналитик должен читать не только результат запроса, но и путь, которым СУБД пришла к этому результату.



EXPLAIN и EXPLAIN ANALYZE

```
EXPLAIN
SELECT customer_id, COUNT(*)
FROM orders
WHERE order_date >= DATE '2026-01-01'
GROUP BY customer_id;
```

```
EXPLAIN ANALYZE
SELECT customer_id, COUNT(*)
FROM orders
WHERE order_date >= DATE '2026-01-01'
GROUP BY customer_id;
```

EXPLAIN

- строит план;
- запрос не выполняется;
- показывает оценку стоимости.

EXPLAIN ANALYZE

- выполняет запрос;
- показывает фактическое время;
- требует осторожности для INSERT/UPDATE/DELETE.



Поля плана: cost, rows, width

```
HashAggregate (cost=1842.00..1960.00 rows=12000 width=40)
  Group Key: customer_id
    -> Seq Scan on orders (cost=0.00..1540.00 rows=60400 width=16)
      Filter: (order_date >= '2026-01-01'::date)
```

- cost — относительная оценка стоимости, не миллисекунды;
- rows — ожидаемое число строк на выходе узла;
- width — ожидаемый средний размер строки;
- от точности rows зависит выбор JOIN, сортировки и агрегации.

Фактические значения в EXPLAIN ANALYZE

```
HashAggregate (actual time=42.121..45.337 rows=11892 loops=1)
  Group Key: customer_id
    -> Seq Scan on orders
      (actual time=0.018..24.712 rows=60231 loops=1)
        Filter: (order_date >= '2026-01-01'::date)
```

Как читать

Сравниваем оценку rows с фактическим rows. Сильное расхождение указывает на проблему статистики, селективности условия или корреляции признаков.

$$\text{ошибка оценки} = \frac{\text{actual rows}}{\text{estimated rows}}$$



Узлы сканирования таблиц

Узел плана	Смысл
Seq Scan	последовательное чтение таблицы целиком или значительной части
Index Scan	доступ через индекс с чтением строк таблицы
Index Only Scan	чтение только индекса, если все нужные поля есть в индексе
Bitmap Heap Scan	комбинированный доступ: сначала bitmap по индексу, затем чтение страниц таблицы

Практическое правило

Seq Scan не всегда плох. Для чтения большей доли таблицы он может быть дешевле множества случайных обращений через индекс.

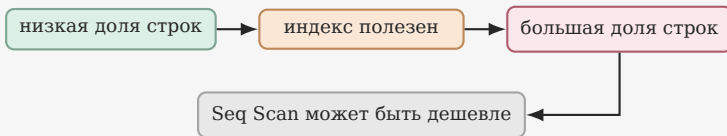


Селективность условия

Определение

Селективность — доля строк таблицы, удовлетворяющих условию фильтрации.

$$s = \frac{|\{r \in T : predicate(r) = true\}|}{|T|}$$



Индекспригодность условия

Лучше для индекса

```
WHERE order_date >= DATE '2026-01-01'  
AND order_date < DATE '2027-01-01'
```

Хуже для обычного индекса

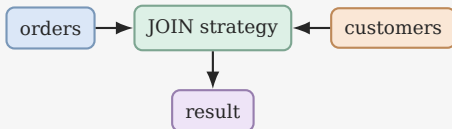
```
WHERE EXTRACT(YEAR FROM order_date) = 2026
```

Принцип

Если функция применяется к индексируемому столбцу, оптимизатору часто труднее использовать обычный B-tree индекс. Лучше формулировать условие как диапазон по исходному значению.

Типы JOIN в физическом плане

Узел	Когда типично применяется
Nested Loop	маленькая внешняя таблица, индексный доступ к внутренней
Hash Join	равенство ключей, достаточно памяти для hash-таблицы
Merge Join	входы уже отсортированы или сортировка оправдана



Агрегации и сортировки

Узел	Смысл
Sort	сортировка строк по ключу
HashAggregate	агрегация через hash-таблицу групп
GroupAggregate	агрегация по уже отсортированному потоку
WindowAgg	вычисление оконных функций

Связь с оконными функциями

Оконные функции почти всегда требуют порядка внутри partition. Поэтому в плане часто появляется Sort перед WindowAgg.



Почему оконные функции могут быть дорогими



Стоимость определяется:

- размером входного набора после WHERE;
- числом и размером partition;
- необходимостью сортировки;
- памятью для сортировки и временными файлами;
- количеством окон с разными PARTITION BY/ORDER BY.

Пример. План оконного запроса

```
EXPLAIN ANALYZE
SELECT
  customer_id,
  order_id,
  order_date,
  ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY order_date
  ) AS rn
FROM orders;
```

```
WindowAgg
-> Sort
   Sort Key: customer_id, order_date
-> Seq Scan on orders
```

Если таблица велика, сортировка по customer_id, order_date может стать основной стоимостью.

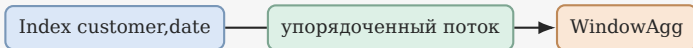


Индекс под оконный запрос

```
CREATE INDEX idx_orders_customer_date  
ON orders(customer_id, order_date);
```

Идея

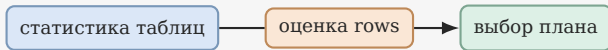
Если индекс соответствует ключам PARTITION BY и ORDER BY, СУБД может уменьшить стоимость сортировки или заменить её чтением данных в подходящем порядке.



Статистика и кардинальность

Определение

Кардинальность — число строк, которое оптимизатор ожидает получить после операции фильтрации, соединения или агрегации.



Если статистика устарела, оптимизатор может выбрать неэффективный JOIN, неверную стратегию агрегации или переоценить пользу индекса.

Чек-лист чтения EXPLAIN

- 1 Какая таблица читается первой и каким способом?
- 2 Есть ли Seq Scan по большой таблице?
- 3 Совпадают ли ожидаемые и фактические rows?
- 4 Какой тип JOIN выбран?
- 5 Где появляются Sort, HashAggregate, WindowAgg?
- 6 Используются ли индексы, созданные под запрос?
- 7 Есть ли повторные циклы loops с большим числом строк?
- 8 Какая операция даёт основное время?



От анализа SQL к распределённой обработке

Переход

Навык чтения плана запроса нужен не только в PostgreSQL. В Spark, ClickHouse, Hive, Trino и других системах аналитик также должен понимать физический план выполнения.



В третьей лекции понятия scan, join, aggregate, sort, exchange и shuffle будут рассмотрены уже в распределённом исполнении.



Раздел

NoSQL

Модели хранения за пределами классической
таблицы

Почему реляционной модели бывает недостаточно

- данные имеют вложенную структуру: заказ содержит позиции, адрес, платежи, события;
- схема меняется часто и не всегда синхронно с приложением;
- запись идёт с очень высокой скоростью;
- требуется горизонтальное масштабирование по множеству узлов;
- данные естественно являются графом связей;
- важен полнотекстовый поиск, а не табличная агрегация;
- временные ряды имеют особые паттерны записи и хранения.

Тезис

NoSQL возник не как «отмена SQL», а как ответ на разнообразие моделей данных и эксплуатационных требований.



Что означает NoSQL

Определение

NoSQL-системы — семейство систем хранения, в которых данные не обязательно представлены в виде нормализованных таблиц, а модель доступа часто проектируется под конкретные запросы и масштабирование.

Key-Value

Document

Wide-column

Search

Graph

Time-series

Корректнее читать NoSQL как «not only SQL»: рядом с SQL появляются другие модели доступа.



NoSQL — это не одна технология

Модель	Единица хранения	Классическая задача
Key-Value	ключ → значение	получить объект по ключу
Document	JSON/BSON-документ	прочитать вложенный объект
Wide-column	строка + семейства колонок	читать по partition key
Graph	вершины и рёбра	пройти связи и путь
Search	инвертированный индекс	найти документы по части слов
Time-series	метрика во времени	агрегировать значения по окну времени

Практический вывод

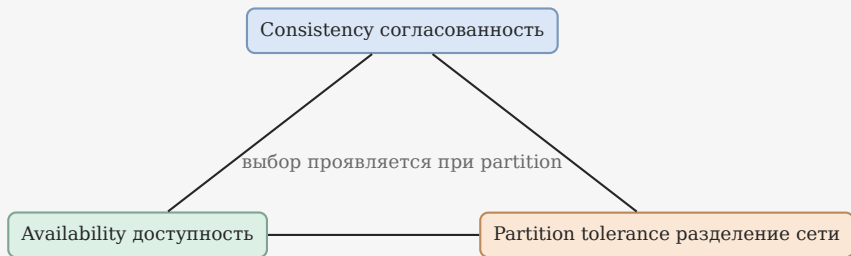
Вопрос «какая NoSQL база лучше?» некорректен без описания данных, запросов, объёма, частоты записи и требований к согласованности.



САР как инженерное ограничение

Идея САР

В распределённой системе при сетевом разделении невозможно одновременно гарантировать строгую согласованность всех ответов и доступность каждого запроса.



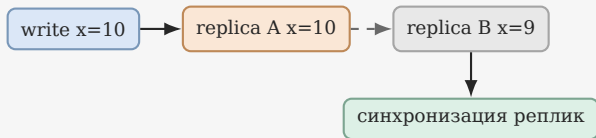
*САР не означает, что система всегда выбирает только две буквы.
Тонкость именно в поведении при сетевом сбое.*



BASE и eventual consistency

Определение

Eventual consistency означает, что при отсутствии новых обновлений реплики со временем сходятся к одному состоянию.

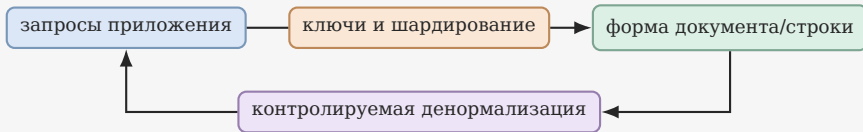


- выигрываем доступность и скорость записи;
- допускаем временное расхождение копий;
- проектируем приложение с учётом конфликтов и повторов.

Моделирование данных под запросы

Принцип NoSQL

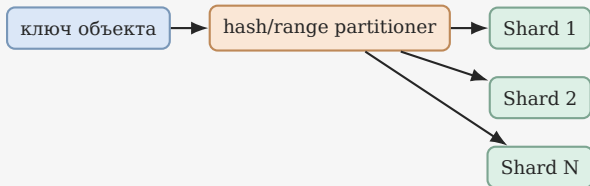
В реляционной модели сначала часто проектируют сущности и связи. В NoSQL сначала фиксируют основные **паттерны доступа**, а именно какие запросы должны быть быстрыми.



Ошибка проектирования ключа может быть дороже отсутствующего индекса.



Горизонтальное масштабирование в NoSQL



Определение

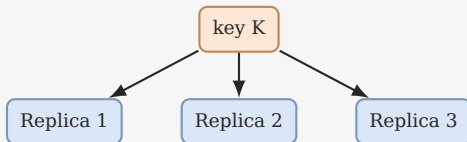
Sharding (шардинг) — распределение данных между узлами по ключу или диапазону ключей для параллельного хранения и обработки запросов.

Главная опасность — **hot partition**. Значительная доля нагрузки попадает в один шард.

Репликация и кворум

Идея

Данные хранятся в нескольких копиях. Для операции чтения и записи система может требовать подтверждения от части реплик.



Если N — число реплик, W — подтверждения записи, R — подтверждения чтения, то условие

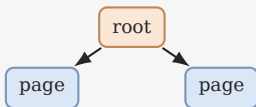
$$W + R > N$$

повышает шанс пересечения чтения с последней успешной записью.

В-tree и LSM-tree как разные стратегии хранения

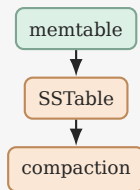
В-tree

Хорош для точечного поиска и диапазонов. Часто используется в реляционных СУБД и части документных систем.



LSM-tree

Оптимизирует запись. Данные сначала попадают в память и журнал, затем сбрасываются и компактизируются.



Многие тяжелые для записи NoSQL-системы используют LSM-подобные подходы.

Индексы в NoSQL

- первичный ключ определяет основной быстрый путь доступа;
- вторичные индексы ускоряют дополнительные запросы, но усложняют запись;
- индекс может быть локальным по шардам или глобальным по кластеру;
- глобальные индексы повышают стоимость координации;
- слишком много индексов ухудшает рабочую нагрузку при тяжелой записи;
- в поисковых системах индекс является центральной структурой хранения.

Практический вывод

NoSQL-проектирование начинается не с «создадим индекс на всё», а с анализа путей доступа, частоты операций и распределения ключей.



Раздел

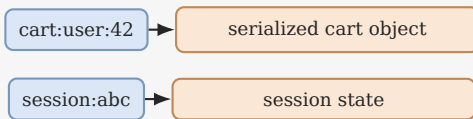
Модели NoSQL

Key-Value, Document, Wide-column, Graph, Search,
Time-series

Key-Value модель

Определение

Key-Value store хранит пары (*key* : *value*). Система эффективно получает значение по ключу, но обычно мало знает о внутренней структуре value.



Примеры: Redis, DynamoDB в key-value сценариях, Riak-подобные системы.

Когда использовать Key-Value

Подходит

- кэширование;
- сессии пользователей;
- корзины интернет-магазина;
- счётчики и ограничения числа запросов;
- быстрый поиск по идентификатору.

Плохо подходит

- сложные JOIN;
- ad-hoc аналитика;
- запросы по произвольным полям значений;
- строгие многострочные транзакции;
- графовые обходы.

Пример

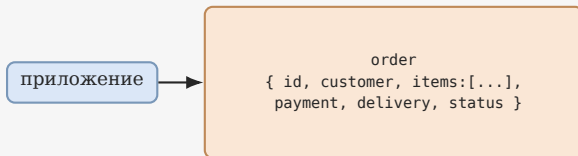
Для проверки авторизации по `session_id` важно получить объект за миллисекунды. Структурная аналитика по всем сессиям — уже другая задача.



Document модель

Определение

Document store хранит документы с вложенной структурой, обычно в формате JSON/BSON или близком представлении.



Примеры: MongoDB, Couchbase, CouchDB, документные режимы мультимодельных СУБД.



Пример документа заказа

```
{
  "order_id": "o_001",
  "customer": {"id": "c_001", "city": "Kazan"},
  "items": [
    {"product_id": "p_01", "price": 1200, "qty": 1},
    {"product_id": "p_17", "price": 500, "qty": 2}
  ],
  "payment": {"type": "card", "status": "paid"},
  "delivery": {"status": "delivered"}
}
```

Идея моделирования

Если заказ обычно читается целиком, вложение позиций внутрь документа уменьшает число обращений и убирает JOIN из критического пути.



Сценарии применения документной модели

- пользовательские профили с разной структурой атрибутов;
- каталоги товаров с неодинаковыми свойствами;
- события web/mobile-приложения;
- заказы и документы, читаемые целиком;
- прототипы продуктов с быстро меняющейся схемой;
- хранение JSON API payload для последующей обработки.

Ограничение

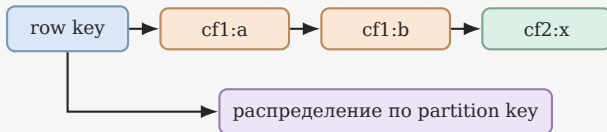
Документная модель удобна для агрегатов приложения, но при сложной отчётности часто требуется отдельная аналитическая витрина или выгрузка в DWH/Data Lake.



Wide-column модель

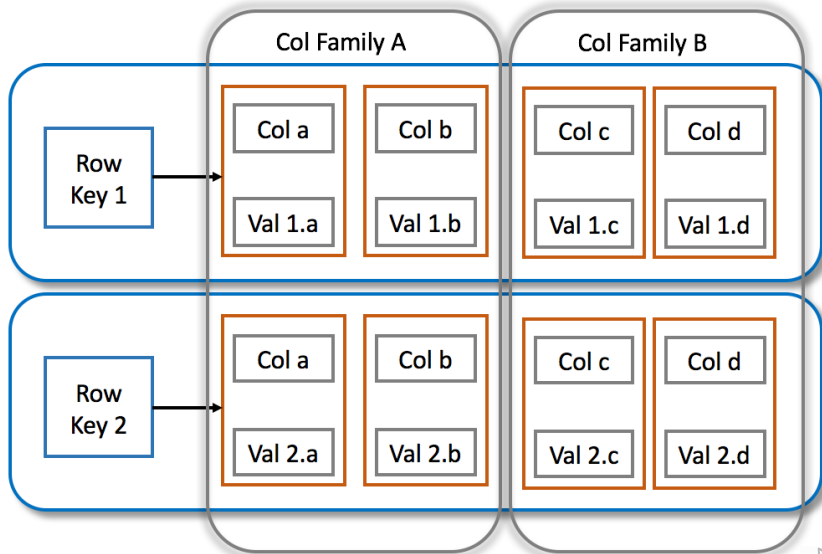
Определение

Wide-column store хранит данные как распределённые строки, сгруппированные по ключам и семействам колонок. Модель оптимизирована под масштабируемую запись и чтение по заранее заданным ключам.



Примеры: Apache Cassandra, Apache HBase, ScyllaDB.

Схематично wide-column модель



Когда полезна wide-column модель

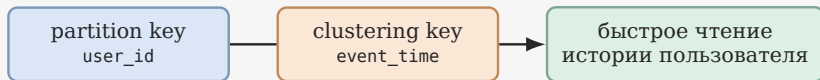
- большие потоки событий;
- телеметрия устройств;
- журналы действий пользователей;
- запись с высокой пропускной способностью;
- чтение по ключу объекта и диапазону времени;
- хранение историй состояний.

Типичный запрос

Получить события пользователя `user_id = 42` за последние 24 часа. Под такой запрос заранее проектируется `partition key` и `clustering order`.



Проектирование ключа wide-column модели

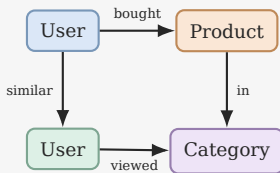


- partition key отвечает за распределение данных по узлам;
- clustering key отвечает за порядок внутри partition;
- неверный ключ приводит к hot partitions или невозможности эффективного запроса;
- JOIN-подход из SQL здесь не является основным способом моделирования.

Graph модель

Определение

Graph database хранит данные как вершины, рёбра и свойства. Основная операция — обход связей между объектами.



Примеры: Neo4j, JanusGraph, Amazon Neptune, ArangoDB в графовом режиме.

Пример визуализации Neo4j

Query:
`match (n:Crew)-[r:friend]->(m) return n.name, m.name`

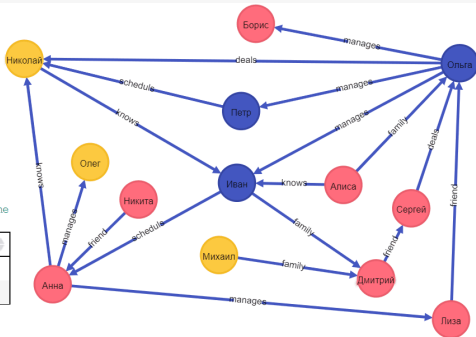
n.name	m.name
Лиза	Ольга
Никита	Анна
Дмитрий	Сергей

Query took 1 ms and returned 3 rows. [Result Details](#)

Query:
`match (n{name: 'Ольга'})-[r:deals]->(m) return n.name, m.name`

n.name	m.name
Ольга	Сергей
Ольга	Николай

Query took 7 ms and returned 2 rows. [Result Details](#)



Прикладные сценарии графовой модели

- рекомендации: «пользователи, похожие на вас, покупали...»;
- fraud detection: подозрительные цепочки платежей и аккаунтов;
- социальные графы и сообщества;
- knowledge graph для связанных сущностей;
- маршруты, зависимости, lineage данных;
- анализ влияния: какие компоненты пострадают при сбое.

Почему не просто SQL JOIN

Глубокие и переменные обходы графа через многократные JOIN становятся громоздкими. Графовая модель делает связи первоклассным объектом хранения.



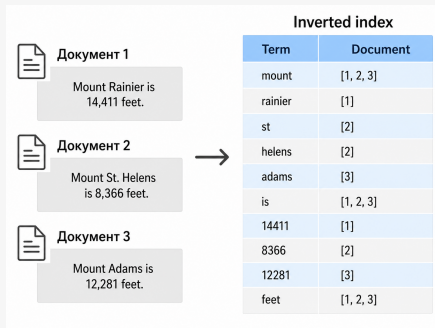
Search / inverted index модель

Определение

Поисковая система хранения строит инвертированный индекс. Индекс строится от термина или признака к списку документов, где он встречается.



Примеры: Elasticsearch, OpenSearch, Apache Solr.



Сценарии применения поисковой модели

- поиск по каталогу товаров;
- анализ логов и событий;
- полнотекстовый поиск по документам;
- фильтры, фасеты, ранжирование результатов;
- близкая к real-time аналитика по событиям;
- визуальные мониторинговые пайплайны для логов.

Важно

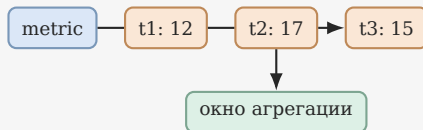
Поисковая система может выполнять агрегации, но её главный профиль — индексирование, поиск и фильтрация, а не замена транзакционной СУБД.



Time-series модель

Определение

Time-series database оптимизирует хранение последовательностей измерений, привязанных ко времени, метрике и набору меток и тегов.



Примеры: InfluxDB, Prometheus как мониторинговая TSDB, VictoriaMetrics, OpenTSDB.

Сценарии применения модели временных рядов

- мониторинг CPU, RAM, сети и дисков;
- метрики приложений и микросервисов;
- IoT-телеметрия;
- финансовые котировки и торговые события;
- технологические датчики промышленности;
- временные агрегаты и алерты.

Особенность

Главная ось данных — время. Поэтому важны retention policy, downsampling, агрегаты по окнам и быстрый диапазонный доступ.



Сравнение моделей NoSQL

Модель	Сильная сторона	Типовые системы	Ограничение
Key-Value	быстрый доступ по ключу	Redis, DynamoDB	слабая ad-hoc аналитика
Document	гибкая вложенная структура	MongoDB, Couchbase	сложные JOIN и отчётность
Wide-column	масштабируемая запись	Cassandra, HBase, ScyllaDB	запросы надо проектировать заранее
Graph	обход связей	Neo4j, JanusGraph	сложнее горизонтально масштабировать
Search	полнотекстовый поиск	Elasticsearch, OpenSearch, Solr	не транзакционная OLTP-замена
Time-series	метрики во времени	InfluxDB, Prometheus	специализированная модель

Вывод

NoSQL — это набор инженерных компромиссов, а не универсальная альтернатива реляционным СУБД.



Раздел

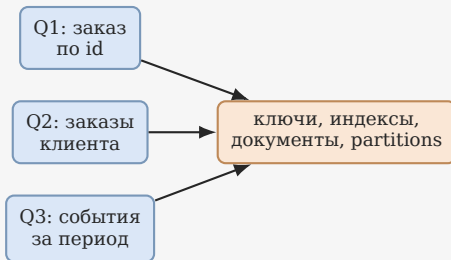
Принципы хранения в NoSQL

Ключи, партиции, репликация, согласованность, денормализация

Принцип 1. Access pattern first

Формулировка

Сначала описываются запросы и ограничения времени ответа, затем выбираются ключи, индексы и форма хранения.



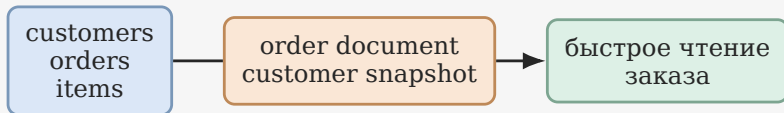
В NoSQL плохая модель хранения не исправляется одним универсальным JOIN.



Принцип 2. Контролируемая денормализация

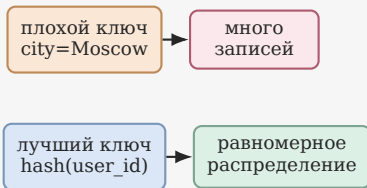
Определение

Денормализация — намеренное дублирование данных ради ускорения чтения, упрощения запроса или устранения распределённого JOIN.



- выигрываем скорость чтения;
- проигрываем простоту обновления;
- требуется стратегия синхронизации копий;
- аналитике нужно знать, какие поля являются снимками состояния.

Принцип 3. Shard key определяет судьбу системы

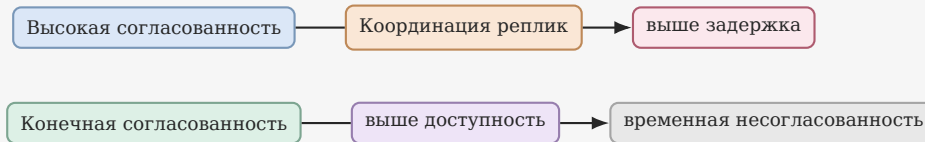


Критерии хорошего ключа

Высокая кардинальность, равномерность распределения, соответствие частым запросам, отсутствие одной доминирующей группы, возможность диапазонного доступа там, где он действительно нужен.



Принцип 4. Согласованность имеет цену



Инженерный выбор

Для платежей и остатков нужна более строгая модель. Для ленты событий, просмотров и телеметрии часто допустима конечная согласованность.

Принцип 5. Запись и чтение конкурируют

Оптимизация записи

- журналы только для вставки данных;
- LSM-структуры;
- пакетная запись;
- меньше вторичных индексов;
- конечная согласованность.

Оптимизация чтения

- материализованные представления;
- индексы;
- денормализация;
- кэши;
- предварительные агрегаты.

Вывод

Нельзя одновременно бесплатно получить максимальную скорость записи, произвольную аналитику и строгую согласованность на всех узлах.



Принцип 6. Гибкая схема не означает доступ без схемы

Тезис

Отсутствие строгой табличной схемы не отменяет структуры данных. Схема просто переносится из DDL в приложение, JSON Schema, контракт API, каталог данных или ETL-процесс.

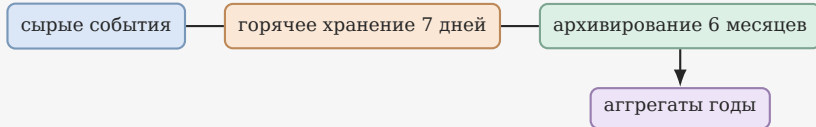


Если схему не описать явно, её всё равно придётся восстанавливать при аналитике.

Принцип 7. TTL и жизненный цикл данных

Определение

TTL — time to live, правило автоматического удаления или устаревания данных после заданного времени.



В логах, метриках и событиях часто важнее сохранить агрегаты, чем бесконечно хранить каждое сырое событие.



Принцип 8. Операционная БД не равна аналитической



Вывод

Даже если NoSQL-система поддерживает агрегации, тяжёлую историческую аналитику часто выносят в отдельный слой, Spark, DWH, lakehouse или специализированную OLAP-систему.

Раздел

Аналитика данных с NoSQL

Как NoSQL включается в Big Data pipeline

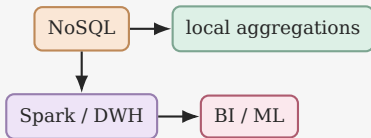
Два режима использования NoSQL в аналитике

Встроенная аналитика

Агрегации выполняются внутри самой системы: агрегация документов, агрегации по поисковым запросам, запросы на временные ряды.

Внешняя аналитика

Данные выгружаются или читаются Spark/DWH для тяжёлой обработки, ML, BI и долгой истории.



Почему тяжёлая аналитика не всегда должна жить в NoSQL

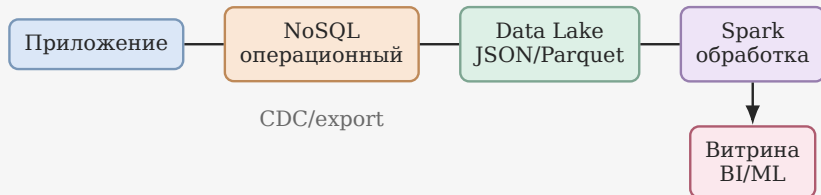
- операционная система оптимизирована под текущую нагрузку приложения;
- аналитические запросы могут сканировать большие исторические объёмы;
- ad-hoc запросы плохо предсказуемы;
- BI требует стабильных витрин и согласованных метрик;
- ML требует инженерии признаков, слияния с внешними источниками и воспроизводимости;
- аналитика не должна ухудшать задержку пользовательских операций.

Архитектурное правило

Операционное NoSQL-хранилище часто является источником для аналитического слоя, а не единственным местом анализа.



NoSQL + Data Lake + Spark



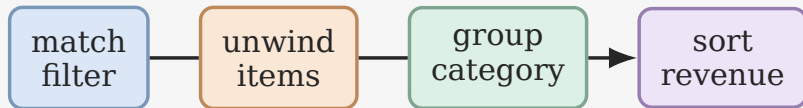
Идея

NoSQL хранит оперативные данные приложения. Data Lake сохраняет историю в аналитически удобных форматах. Spark выполняет очистку, агрегации и подготовку витрин.

Document analytics: агрегационный pipeline

Идея

Документные БД обычно поддерживают конвейер преобразований: фильтрация документов, разворачивание массивов, группировка, сортировка, проекция результата.



Это удобно для оперативных отчётов по документам, но не отменяет отдельный DWH/Data Mart для корпоративных метрик.



Wide-column аналитика. Ограничения и подходы

Что удобно

- чтение по partition key;
- диапазоны по времени;
- предрасчитанные таблицы под запрос;
- быстрые записи событий.

Что сложнее

- произвольные JOIN;
- полный scan всей истории;
- ad-hoc группировки;
- частая смена аналитических вопросов.

Практическая схема

Для тяжёлой аналитики wide-column данные часто читаются Spark-коннектором или выгружаются в lakehouse/Parquet.



Search аналитика. Логи, события, фасеты



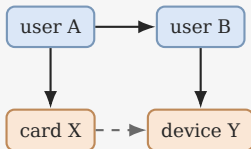
- фильтрация по сервису, уровню ошибки, пользователю;
- агрегации по временным интервалам;
- top-N ошибок и endpoint-ов;
- быстрый поиск по тексту лога;
- полезно для мониторинга и расследования инцидентов.



Graph аналитика. Связи как данные

Типовые аналитические задачи

- найти кратчайший путь между сущностями;
- измерить центральность узла;
- обнаружить сообщества;
- найти подозрительные циклы и цепочки;
- построить рекомендации на графе взаимодействий.



Time-series аналитика. Окно времени как основной оператор

Схема запроса

Метрика выбирается по имени иметке, затем агрегируется по временным окнам.

$$\text{avg}_{5m}(x) = \frac{1}{|W_t|} \sum_{i \in W_t} x_i, \quad W_t = [t - 5m, t]$$

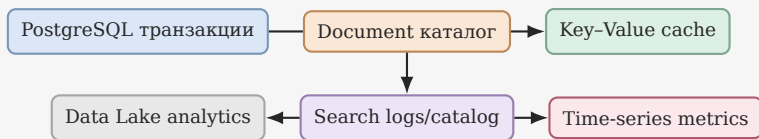
- средняя задержка за 5 минут;
- число ошибок в минуту;
- рост потребления памяти;
- процентиль задержки;
- алерт при превышении порога.



Polyglot persistence

Определение

Polyglot persistence — архитектурный подход, при котором разные части системы используют разные модели хранения, соответствующие своим данным и нагрузкам.



Зрелая архитектура не пытается решить все задачи одной СУБД.

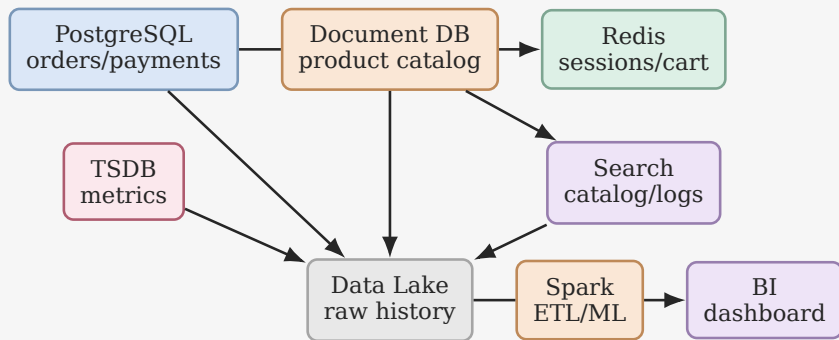


Выбор модели хранения

Сценарий	Предпочтительная модель	Почему
Заказы и платежи	Реляционная / транзакционная	целостность, ACID, SQL
Каталог товаров с разными атрибутами	Document	гибкая вложенная структура
Сессии и кэш	Key-Value	быстрый доступ по ключу
Клики и телеметрия событий	Wide-column stream+lake	/ высокая запись, масштаб
Поиск по товарам и логам	Search	инвертированный индекс
Связи пользователей и рекомендаций	Graph	обход отношений
Метрики инфраструктуры	Time-series	окна времени, retention
Корпоративная отчётность	DWH / Data Mart	согласованные показатели

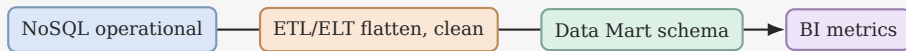


End-to-end пример. Интернет-магазин



Одна предметная область порождает несколько моделей данных и несколько систем хранения.

NoSQL и BI: почему нужна витрина

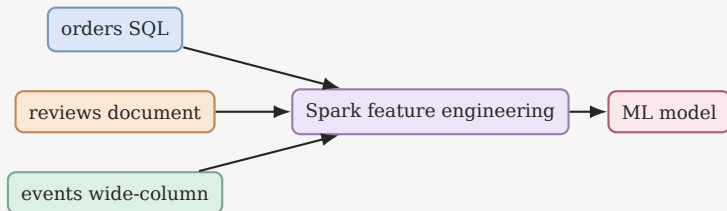


BI-инструментам нужны:

- стабильные поля;
- понятные dimensions и measures;
- согласованные агрегаты;
- контролируемая история изменений;
- приемлемое время ответа.



NoSQL и ML: признаки из разных хранилищ



Тезис

Машинное обучение часто требует собрать признаки из транзакций, документов, событий, логов и внешних источников. Поэтому storage layer почти всегда полиглотен.

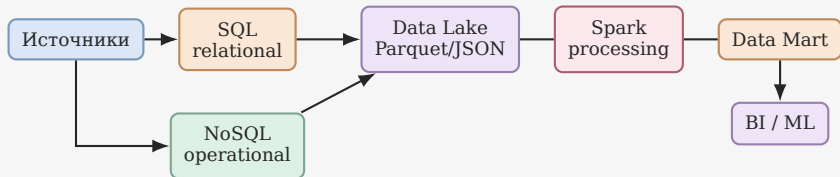


Раздел

Итоги

Как выбрать и анализировать систему хранения

Сводная архитектурная схема



Главный вывод

Хранение данных — это не пассивный слой. Выбор модели хранения заранее определяет допустимые запросы, стоимость аналитики и сложность масштабирования.



Контрольные вопросы

- 1 Почему DDL, DML и TCL отражают разные уровни работы с БД?
- 2 Чем оконная функция отличается от GROUP BY?
- 3 Зачем в оконной функции нужны PARTITION BY, ORDER BY и frame?
- 4 Почему EXPLAIN ANALYZE может изменить данные при неосторожном использовании?
- 5 Что означают Seq Scan, Index Scan, Hash Join, WindowAgg?
- 6 Почему NoSQL нельзя сводить к одной технологии?
- 7 Как выбрать между document, key-value, wide-column и graph model?
- 8 Почему NoSQL-хранилище часто не является конечным BI-слоем?



Спасибо за внимание!